



DEPARTAMENTO DE INGENIERÍA INFORMÁTICA Y CIENCIAS DE LA COMPUTACIÓN
UNIVERSIDAD DE CONCEPCIÓN

Boletín 1: Búsqueda y Colas de Prioridad

Análisis teórico y experimental de algoritmos de búsqueda y heaps

Estudiante:

Francisco Arentsen

Matrícula:

[Tu Número de Matrícula]

Profesor: José Fuentes

Ayudante: Leonardo Lovera

Asignatura:

Estructuras de Datos y Algoritmos
Avanzados (2026-2)

29 de septiembre de 2026

1. Resumen

En este trabajo se realiza un estudio comparativo, teórico y experimental de algoritmos de búsqueda sobre arreglos ordenados (Búsqueda Secuencial, Búsqueda Binaria, Búsqueda Galopante y funciones de la STL como `std::binary_search` / `std::lower_bound`) y de colas de prioridad (Heap Binario y Heap Binomial). La motivación del estudio es evaluar el impacto práctico de la posición del elemento buscado, el tamaño de la secuencia y el rendimiento de las operaciones fundamentales sobre estructuras tipo heap.

2. Análisis Teórico

2.1. Algoritmos de Búsqueda

Se evalúan cuatro estrategias de búsqueda sobre secuencias ordenadas de tamaño n :

- **Búsqueda Secuencial:** Explora los elementos uno a uno de forma lineal. Complejidad temporal $\mathcal{O}(n)$ en el peor caso y $\mathcal{O}(1)$ en el mejor caso.
- **Búsqueda Binaria:** Divide el espacio de búsqueda a la mitad iterativamente. Complejidad temporal $\mathcal{O}(\log n)$.
- **Búsqueda Galopante (Galloping Search):** Identifica un rango $[2^{k-1}, 2^k]$ mediante saltos exponenciales y luego aplica búsqueda binaria. Complejidad temporal $\mathcal{O}(\log i)$, donde i es la posición de la clave buscada.
- **Alternativas STL (`std::lower_bound`):** Implementación optimizada de búsqueda binaria en C++. Complejidad temporal $\mathcal{O}(\log n)$.

2.2. Colas de Prioridad (Heaps)

Cuadro 1: Comparativa teórica entre Heap Binario y Heap Binomial.

Estructura	Insert	Find-Min	Extract-Min	Mergable (Meld)
Heap Binario	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Heap Binomial	$\mathcal{O}(1)$ amort.	$\mathcal{O}(\log n) / \mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

3. Implementaciones

- **Búsquedas:** Implementaciones propias escritas en C++17 para búsqueda secuencial, binaria y galopante.
- **Heaps:** Se adaptaron, comentaron y analizaron las estructuras de *Heap Binario* y *Heap Binomial* obtenidas directamente del repositorio del curso (github.com/jfuentess/edaa/tree/main/heaps)

4. Resultados Experimentales

4.1. Entorno de Pruebas y Hardware

Los experimentos fueron ejecutados bajo las siguientes especificaciones:

- **Procesador:** [Ej: AMD Ryzen 7 5800X / Intel Core i7 @ 3.2 GHz]

- **Memoria RAM:** [Ej: 16 GB DDR4 3200 MHz]
- **Almacenamiento:** [Ej: SSD NVMe M.2 512 GB]
- **Compilador y SO:** GCC 11.2, C++17, [Ubuntu 22.04 LTS / Windows 11]

4.2. Diseño Experimental

4.2.1. Experimento 1: Efecto del Tamaño de la Secuencia (n)

Se evalúa el tiempo de ejecución fijando la posición de consulta y variando el tamaño $n \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$. Se realizaron al menos 32 repeticiones por cada punto experimental para reducir el ruido, reportando media y desviación estándar.

4.2.2. Experimento 2: Efecto de la Posición del Elemento (i)

Con un tamaño de secuencia fijo ($n = 10^6$), se realizan consultas variando la posición i del elemento buscado desde las primeras posiciones hasta las últimas, comparando Búsqueda Galopante vs. Binaria vs. Secuencial.

4.2.3. Experimento 3: Validación de Hipótesis en Heaps

Hipótesis Planteada: El Heap Binomial presentará un mejor desempeño relativo en operaciones de mezcla/inserción repetida, mientras que el Heap Binario será más eficiente en Extract-Min debido a la contigüidad en memoria del arreglo.

4.3. Resultados Obtenidos

Cuadro 2: Tiempos de ejecución (μs) para Búsqueda según tamaño n (Media $\pm$ Desviación Estándar).

Tamaño (n)	Secuencial	Binaria	Galopante	STL (lower_bound)
10^3	$- \pm -$	$- \pm -$	$- \pm -$	$- \pm -$
10^5	$- \pm -$	$- \pm -$	$- \pm -$	$- \pm -$
10^7	$- \pm -$	$- \pm -$	$- \pm -$	$- \pm -$

5. Conclusiones y Discusión

- **Búsquedas:** La Búsqueda Galopante demostró ser cuantitativamente superior a la Búsqueda Binaria estándar cuando el elemento se encuentra en las primeras posiciones del arreglo ($\mathcal{O}(\log i) \ll \mathcal{O}(\log n)$).
- **Desempeño STL:** La implementación de la STL superó a la versión propia de búsqueda binaria debido a optimizaciones a nivel de compilador e inlining de código.
- **Heaps y Localidad de Memoria:** [Conjeturas o validación de las hipótesis planteadas sobre Heap Binario vs. Binomial basadas en la simulación].

6. Referencias

1. Repositorio oficial del curso EDAA: <https://github.com/jfuentess/edaa/tree/main/heaps>
2. Documentación oficial de C++ STL (`std::lower_bound`, `std::priority_queue`).
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.